

Programming II

Object Oriented Programming (OOP) **Static Class Members**

Static Class Members:

- Static fields and static methods do not belong to a single instance of a class.
- To invoke a static method or use a static field, the class name, rather than the instance name is used.

Example:

double x = Math.sqrt (25.0)



Example:

How do I get the number of objects created from a certain class?

```
public class Car{
    private String maker;
    private int model;
    private int no_of_objects;
    public Car ()
    {
        maker = "Honda";
        model = 2017;
        no_of_objects++;
    }
    public Car (String m , int mo)
    {
        maker = m;
        model = mo;
        no_of_objects++;
    }
    public int get_no_of_objects()
    {
        return no_of_objects;
    }
}
```

Example:

```
public class Main
{
    public static void main(String[] args) {
        Car c1 = new Car();
        Car c2 = new Car();
        Car c3 = new Car();
        Car c4 = new Car();
        Car c5 = new Car();
        System.out.println(c3.get_no_of_objects());
    }
}
```



Example:

```
public class Main
{
    public static void main(String[] args) {
        Car c1 = new Car();
        Car c2 = new Car();
        Car c3 = new Car();
        Car c4 = new Car();
        Car c5 = new Car();
        System.out.println(c5.get_no_of_objects());
    }
}
```



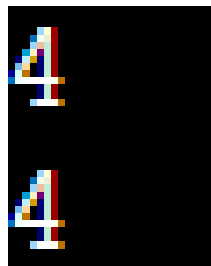
Example (**static** Keyword):

```
public class Car{
    private String maker;
    private int model;
    private static int no_of_objects;
    public Car ()
    {
        maker = "Honda";
        model = 2017;
        no_of_objects++;
    }
    public Car (String m , int mo)
    {
        maker = m;
        model = mo;
        no_of_objects++;
    }
    public static int get_no_of_objects()
    {
        return no_of_objects;
    }
}
```

Example:

```
public class Main
{
    public static void main(String[] args) {
        Car c1 = new Car();
        Car c2 = new Car();
        Car c3 = new Car();
        Car c4 = new Car();

        System.out.println(Car.get_no_of_objects());
        System.out.println(c2.get_no_of_objects());
    }
}
```

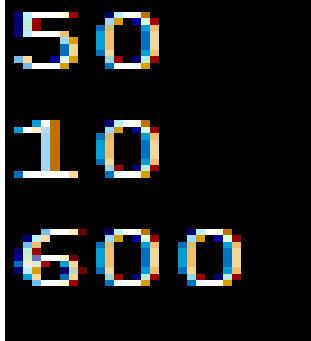


Example 2(Utility Class)

```
public class Calc
{
    public static int add (int a , int b)
    {
        return a + b;
    }
    public static int subtract (int a , int b)
    {
        return a - b;
    }
    public static int multiply (int a , int b)
    {
        return a * b;
    }
}
```

Example 2(Utility Class)

```
public class Main
{
    public static void main(String[] args) {
        System.out.println(Calc.add (30, 20));
        System.out.println(Calc.subtract (30, 20));
        System.out.println(Calc.multiply (30, 20));
    }
}
```

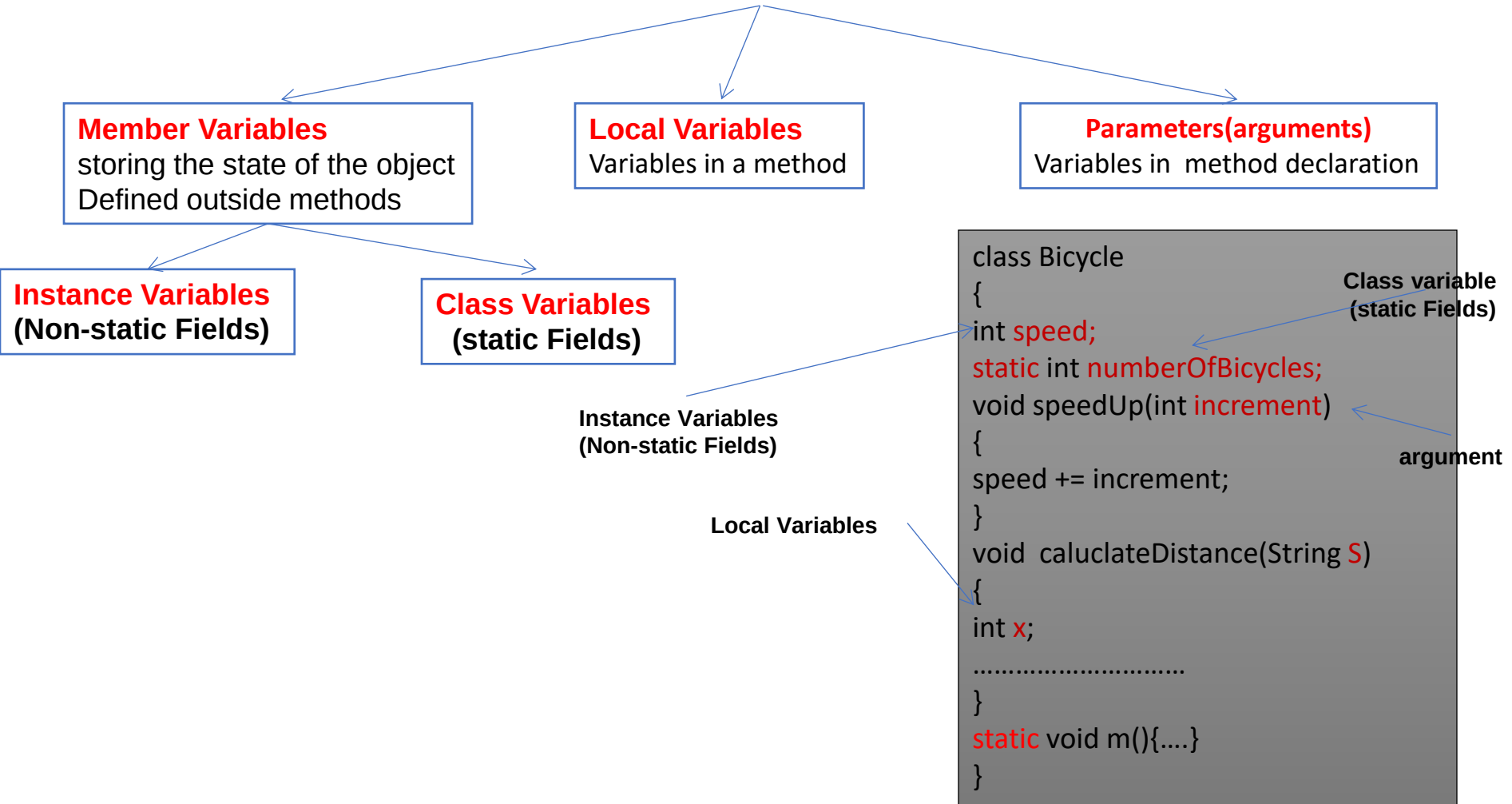


50
10
600

Static Methods:

- Static methods are convenient because they may be called at the class level.
- They are typically used to create utility classes, such as the **Math** class in the Java Standard Library.
- Static methods may not communicate with instance fields, only static fields.

Variables in OOP

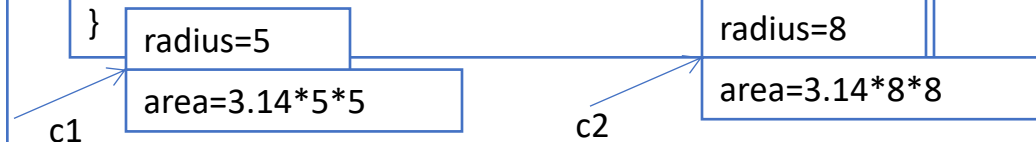


Using this keyword

In OOP (**this**) word always refers to the object that call the method

```
class Circle{
public int radius;
public double area;
public void setArea()
{
this.area=3.14*(this.radius)*(this.radius);
//area=3.14*radius*radius;
}
Circle()
{
}
Circle(int r){
this.radius=r;
//radius=r;}
}
```

```
class Test {
public static void main (String [] arg)
{
Circle c1=new Circle();
c1.radius=5;
c1.setArea();
System.out.println(c1.area);
Circle c2=new Circle(8);
c2.setArea();
System.out.println(c2.area);
}
}
```



➤ Instance variables in instance methods are always related to the **calling object**, JAVA add this. before each of them

Instance Variable Hiding

when a local variable has the same name as an instance variable, the local variable *hides* the instance variable inside the method

```
class A{
    int i=20;
    int j=10;
    void m()
    {
        int i=5;
        System.out.println("i="+i); //prints 5
    }
}
```

```
class A
{
    int i=20;
    int j=10;
    void m()
    {
        int i=5;
        System.out.println("i="+this.i); //prints 20
    }
}
```

```
class Test
{
    public static void main(String [] arg)
    {
        A a=new A();
        a.m();
    }
}
```